PRACTICAL 1

Converting a Machine Learning Notebook into a Flask ML System

AIM: To convert a machine learning model developed in a Jupyter Notebook (.ipynb) into a structured, modular, and deployable Flask-based machine learning application with model serialization, input validation, prediction functionality, and a web interface.

➢ Requirements

- Python 3.10 or 3.11
- Jupyter Notebook or Google Colab
- VS Code or another Python IDE
- Flask, Scikit-learn, Pandas, NumPy, Joblib and pytest

ML system architecture

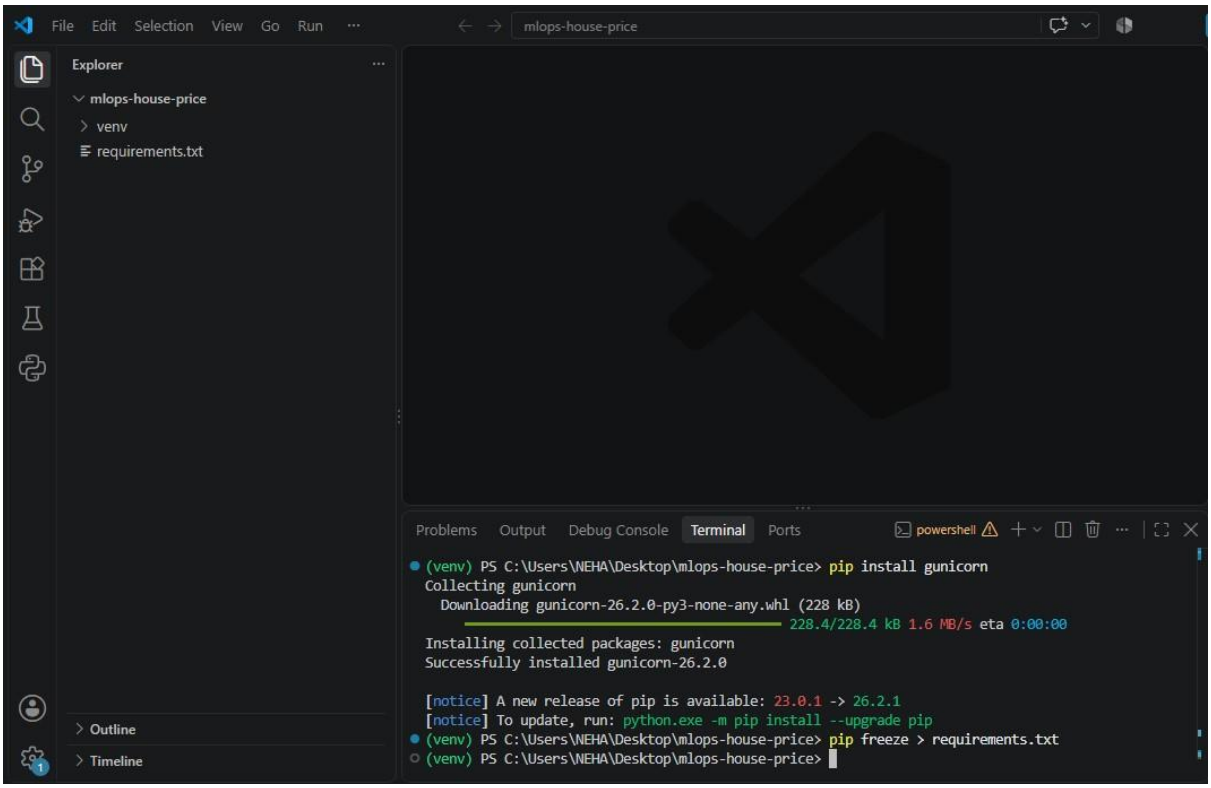
Notebook / training script → trained model pipeline → model.pkl → Flask routes and validation → web interface and REST API.

Step 1 - Create the environment

```
mkdir <Project name>
cd <Project name>
python -m venv venv

.\venv\Scripts\Activate.ps1

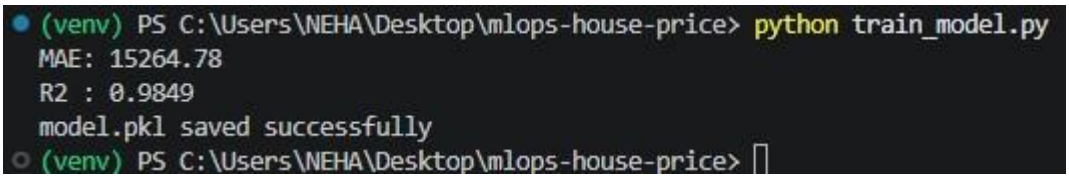
pip install flask scikit-learn pandas numpy joblib pytest
pip install gunicorn
pip freeze > requirements.txt
```



Step 2 - Train and serialize the model

If a suitable fitted pipeline already exists in your notebook, reuse it and preserve its preprocessing steps. Otherwise, create train_model.py.

Expected: the program prints evaluation metrics and creates model.pkl. Metrics vary with the generated data and training process.



Step 3 — Create the project structure

Project File Structure :

For example (Replace it with your project):

```
mlops-house-price/
├── app.py
├── train_model.py
├── model.pkl
├── requirements.txt
├── .gitignore
├── Dockerfile
├── README.md
├── templates/
│   └── index.html
├── static/
```

```
│   └── style.css
└── tests/
    └── test_app.py
```

Step 4 - Develop app.py

Code – app.py

(Create atleast 3 routes. / , /health , /prediction , etc.)

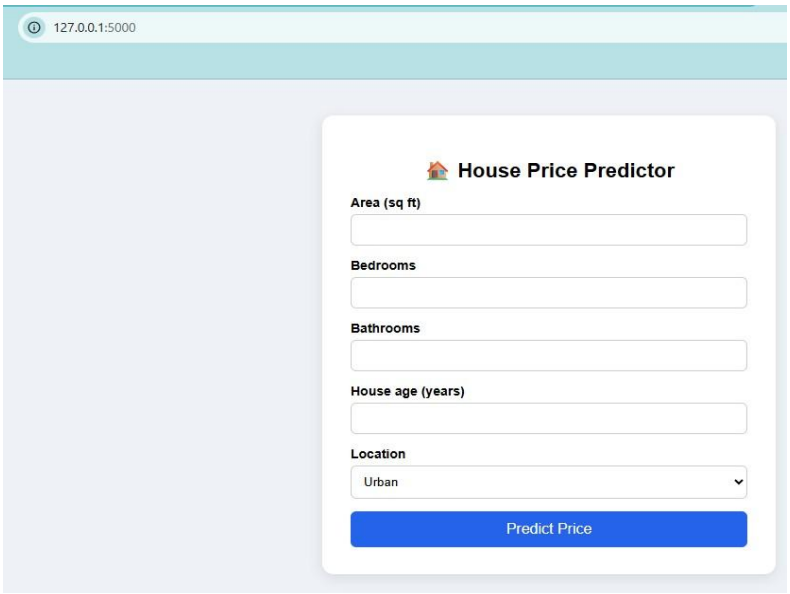
Step 5 – HTML\CSS interface

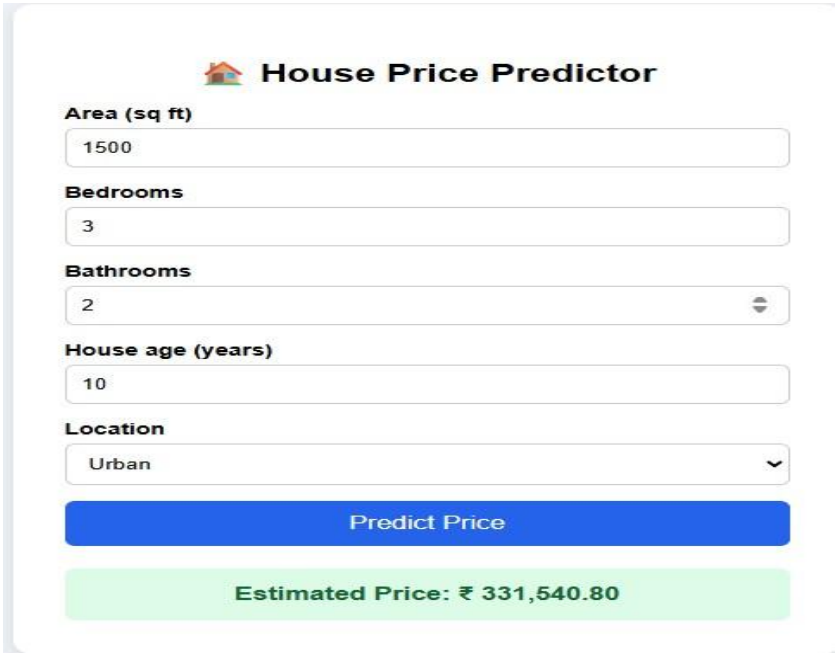
(Create Basic UI)

Step 7 - Run and test the application

```
python app.py
# Open http://127.0.0.1:5000

curl http://127.0.0.1:5000/health
```






```
(venv) PS C:\Users\NEHA\Desktop\mlops-house-price> curl.exe http://127.0.0.1:5000/health
{
  "model_loaded": true,
  "status": "ok"
}
(venv) PS C:\Users\NEHA\Desktop\mlops-house-price> $body = '{"area":1500,"bedrooms":3,"bathrooms":2,"age":10,"location":"urban"}
'
>> Invoke-RestMethod -Uri http://127.0.0.1:5000/api/predict -Method Post -ContentType "application/json" -Body $body | ConvertTo
-Json
{
    "input":  {
                  "age":  10,
                  "area":  1500.0,
                  "bathrooms":  2,
                  "bedrooms":  3,
                  "location":  "urban"
              },
    "predicted_price":  331540.8
}
(venv) PS C:\Users\NEHA\Desktop\mlops-house-price>
```

```
tests\test_app.py ...                                                    [100%]

============================================== 3 passed in 4.79s ===============================================
(venv) PS C:\Users\NEHA\Desktop\mlops-house-price>
```

Step 8 - Create .gitignore

For example :

```
venv/
.venv/
__pycache__/
*.py[cod]
.pytest_cache/
.env
*.pem
```

PRACTICAL 2

Dockerizing the Machine Learning System

AIM: To create a Dockerfile for the Flask-based machine learning application, build a Docker image, run the image as a container, and verify the application's functionality in an isolated and reproducible environment.

➢ Requirements

- Completed Practical 1 project
- Docker Desktop or Docker Engine installed and running
- Flask application and model artifact available

Step 1 — Understand Docker

Dockerfile: instructions for building an image. Docker image: packaged application, model and dependencies. Docker container: a running or stopped instance of an image.

Step 2 — Create the Dockerfile

Step 3 — Create .dockerignore

For example(Replace it if needed):

```
venv/
.venv/
__pycache__/
.git/
.pytest_cache/
.env
*.pem
.ipynb_checkpoints/
```

Step 4 - Build the image

```
docker --version
docker info
```

```
PS C:\Users\NEHA\Desktop\mlops-house-price> docker build -t h
[+] Building 98.3s (10/10) FINISHED
 => [internal] load build definition from Dockerfile
 => => transferring dockerfile: 201B
 => [internal] load metadata for docker.io/library/python:3.1
 => [internal] load .dockerignore
 => => transferring context: 119B
 => [1/5] FROM docker.io/library/python:3.11-slim@sha256:e88e
 => => resolve docker.io/library/python:3.11-slim@sha256:e88e
 => => sha256:496c1a902b3a93cea3e54b90b79dbcaa0e58ad90c3fe9dd
 => => sha256:759a7f30c31818dda4c97b407b24d93da479c306568d185
 => => sha256:0f752a1982d6f819fc2c9e78ac1514c63f2f2e1dec7885f
 => => extracting sha256:0f752a1982d6f819fc2c9e78ac1514c63f2f
 => => extracting sha256:759a7f30c31818dda4c97b407b24d93da479
 => => extracting sha256:496c1a902b3a93cea3e54b90b79dbcaa0e58
 => [internal] load build context
 => => transferring context: 7.32MB
 => [2/5] WORKDIR /app
 => [3/5] COPY requirements.txt .
 => [4/5] RUN pip install --no-cache-dir -r requirements.txt
 => [5/5] COPY . .
 => exporting to image
 => => exporting layers
 => => exporting manifest sha256:52b833c958b8df62a77b29924fba
 => => exporting config sha256:3f66c32a1e2ccb9b1d7d02c1fd350c
 => => exporting attestation manifest sha256:25ddb02374edca17
 => => exporting manifest list sha256:a6814b0a56782396a7ca1f4
 => => naming to docker.io/library/house-price-app:1.0
 => => unpacking to docker.io/library/house-price-app:1.0
```

docker build -t <tag>:1.0 .

docker images

```
PS C:\Users\NEHA\Desktop\mlops-house-price> docker images

IMAGE                  ID             DISK USAGE   CONTENT SIZE   EXTRA
hello-world:latest     11a3eb1597e8       25.9kB         9.49kB   U
house-price-app:1.0    a6814b0a5678        727MB          165MB
nginx:latest           332d0beea97c        243MB         66.8MB
```

Step 5 - Run the container

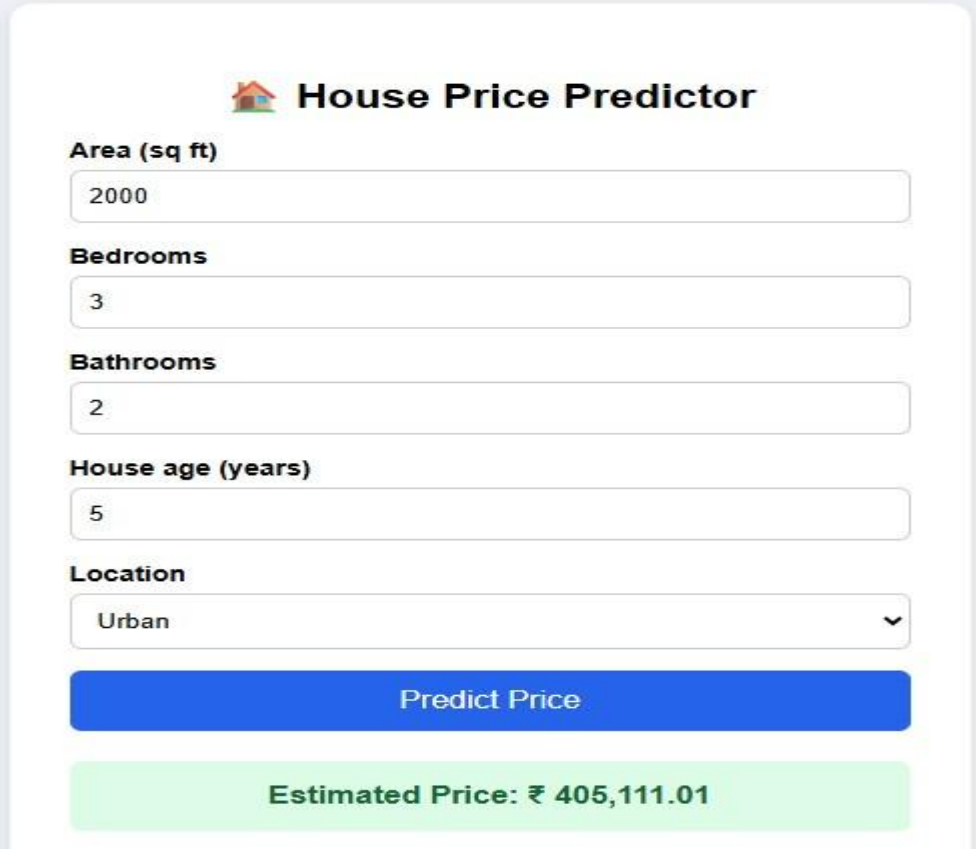
docker run -d --name <name> -p 5000:5000 <tag>:1.0

docker ps

docker logs <name>

Open http://localhost:5000 and test a prediction.

```
PS C:\Users\NEHA\Desktop\mlops-house-price> docker ps
CONTAINER ID   IMAGE              COMMAND          CREATED         STATUS         PORTS                                        NAMES
69f081ec8862   house-price-app:1.0   "python app.py"   8 seconds ago   Up 7 seconds   0.0.0.0:5000->5000/tcp, [::]:5000->5000/tcp   house-price-app
```



Step 6 — Test, stop and remove the container

curl http://localhost:5000/health
docker inspect house-price-app
docker stop house-price-app
docker start house-price-app
docker rm -f house-price-app

```
PS C:\Users\NEHA\Desktop\mlops-house-price> curl.exe http://localhost:5000/health
{"model_loaded":true,"status":"ok"}
PS C:\Users\NEHA\Desktop\mlops-house-price> docker stop house-price-app
house-price-app
PS C:\Users\NEHA\Desktop\mlops-house-price> docker start house-price-app
house-price-app
PS C:\Users\NEHA\Desktop\mlops-house-price> docker rm -f house-price-app
house-price-app
```

PRACTICAL 3

Deploying the Dockerized ML System on AWS EC2

AIM: To deploy the Dockerized machine learning application on an Amazon EC2 virtual machine by configuring an instance, networking and security rules, installing Docker, transferring the application image, and verifying remote access to the deployed system.

➢ Requirements

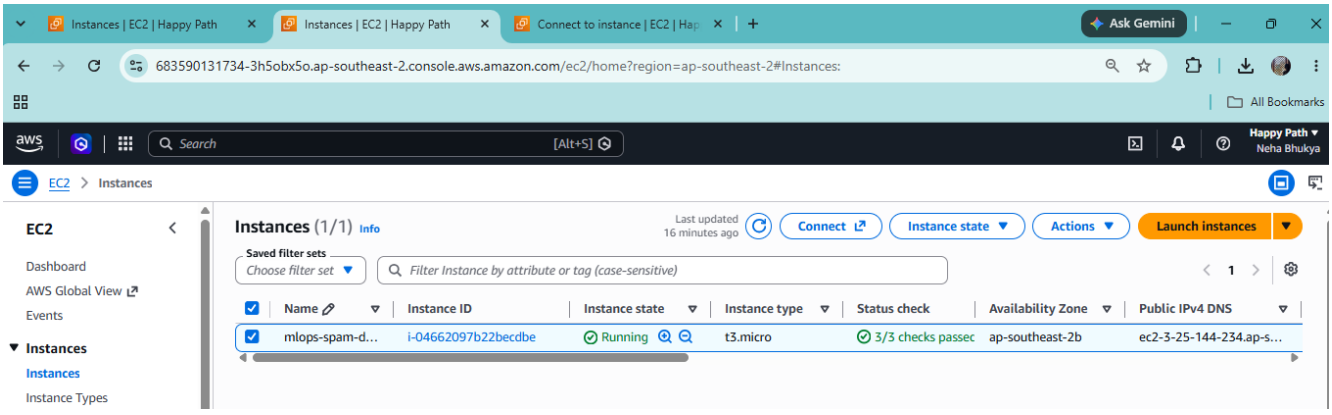
- AWS account with EC2 permissions
- Docker image from Practical 2
- SSH client and a secure key pair
- Small Ubuntu EC2 instance appropriate for the application

Step 1 - Select AWS region

1. Open https://console.aws.amazon.com/ and sign in.
2. Open EC2 and select a region, for example Asia Pacific (Mumbai), ap-south-1.
3. Open the EC2 dashboard.

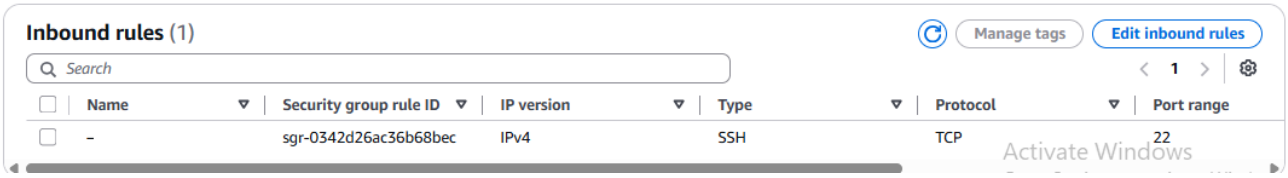
Step 2 - Launch an EC2 instance

4. Select Launch instance and enter mlops-house-price-server as the name.
5. Choose an Ubuntu Server LTS AMI.
6. Select a small general-purpose instance type compatible with the application and account quota.
7. Create or select an SSH key pair and securely store the private key.
8. Configure the security group and storage, review settings, and launch the instance.



Step 3 - Configure the security group

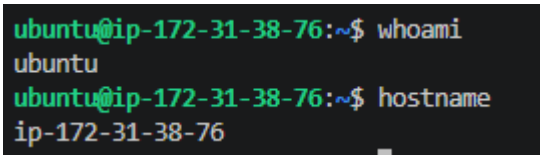
Rule	Port	Source	Purpose
SSH	22	Your public IP /32	Remote administration
Custom TCP	5000	Trusted client range; 0.0.0.0/0 only if public lab access is required	Application access



Step 4 - Connect by SSH

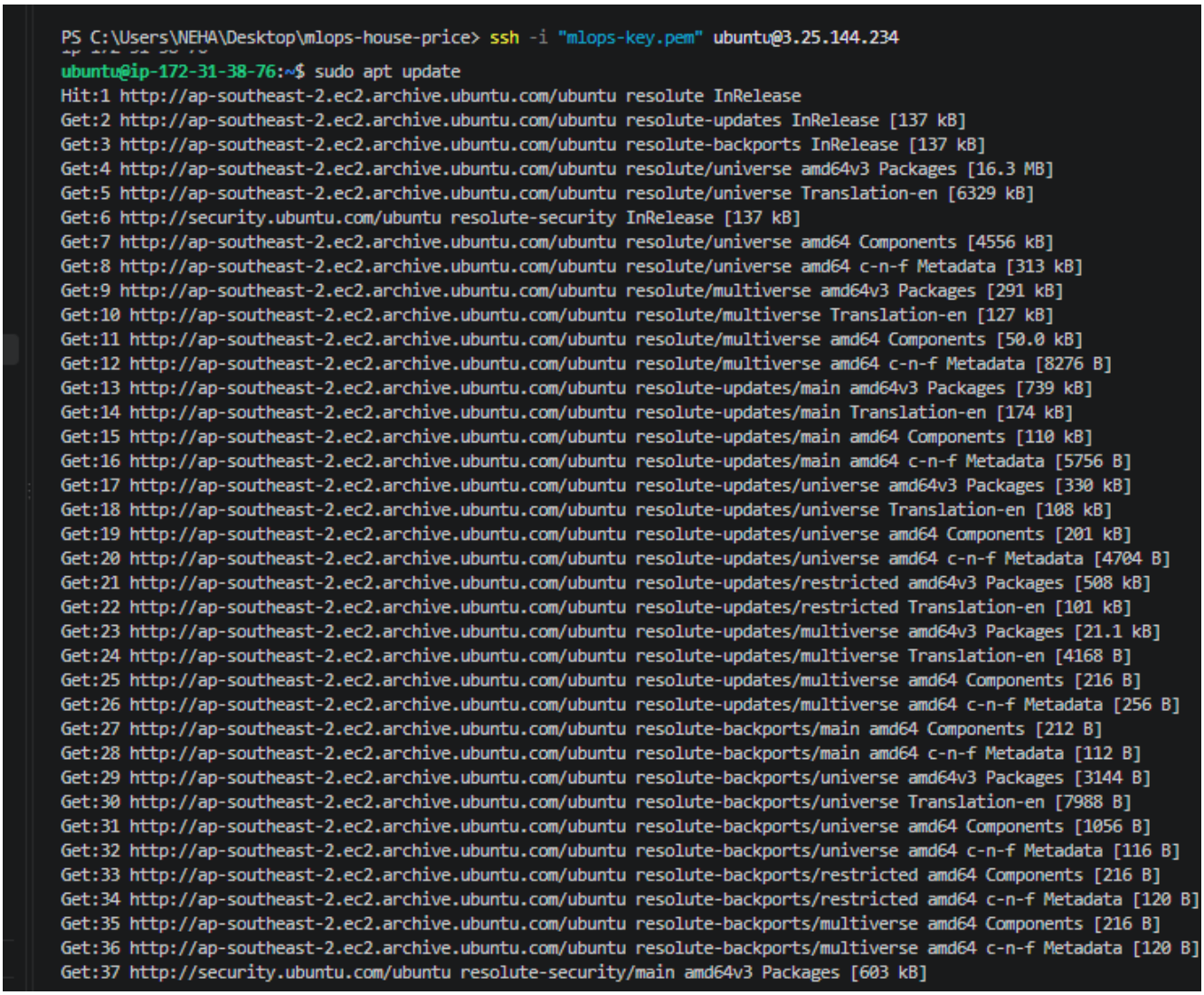
ssh -i <keyname>.pem ubuntu@YOUR_EC2_PUBLIC_IP

Replace YOUR_EC2_PUBLIC_IP with the actual address shown in the console.



Step 5 – Check\Install and Login Docker

sudo apt update



sudo docker –version

```
ubuntu@ip-172-31-38-76:~$ sudo docker --version
Docker version 29.1.3, build 29.1.3-0ubuntu4.1
```

sudo docker info

```
ubuntu@ip-172-31-38-76:~$ sudo docker info
Client:
 Version:    29.1.3
 Context:    default
 Debug Mode: false
 Plugins:
  trust: Manage trust on Docker images (Docker Inc.)
    Version:  29.1.3
    Path:     /usr/libexec/docker/cli-plugins/docker-trust

Server:
 Containers: 0
  Running: 0
  Paused: 0
  Stopped: 0
 Images: 0
 Server Version: 29.1.3
 Storage Driver: overlayfs
  driver-type: io.containerd.snapshotter.v1
 Logging Driver: json-file
 Cgroup Driver: systemd
 Cgroup Version: 2
 Plugins:
  Volume: local
  Network: bridge host ipvlan macvlan null overlay
  Log: awslogs fluentd gcplogs gelf journald json-file local splunk syslog
 CDI spec directories:
  /etc/cdi
  /var/run/cdi
 Swarm: inactive
 Runtimes: io.containerd.runc.v2 runc
 Default Runtime: runc
 Init Binary: docker-init
 containerd version:
 runc version:
 init version:
 Security Options:
  apparmor
```

docker login -u <your_username>

```
ubuntu@ip-172-31-38-76:~$ sudo docker logout
sudo docker login -u 23012531007
Removing login credentials for https://index.docker.io/v1/

i Info → A Personal Access Token (PAT) can be used instead.
         To create a PAT, visit https://app.docker.com/settings


Password:

WARNING! Your credentials are stored unencrypted in '/root/.docker/config.json'.
Configure a credential helper to remove this warning. See
https://docs.docker.com/go/credential-store/

Login Succeeded
```

If not installed;

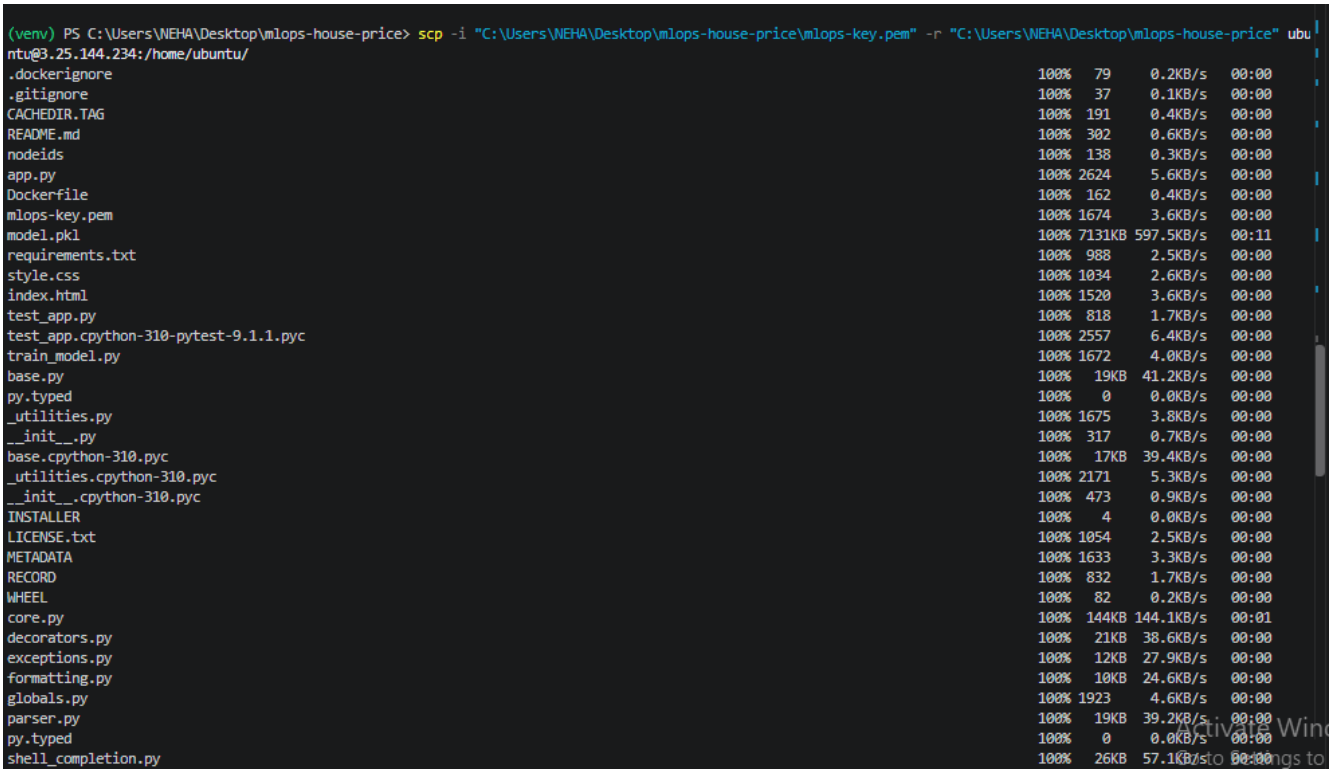
sudo apt install -y docker.io

sudo systemctl enable --now docker

Step 6 - Transfer the Project files to EC2 instance

On your local machine

scp -i "C:\Users\NEHA\Desktop\mlops-house-price\mlops-key.pem" -r "C:\Users\NEHA\Desktop\mlops-house-price" ubuntu@3.25.144.234:/home/ubuntu/



Step 7 — Build Images and Start the container

sudo docker build -t <tag>:1.0 .

```
ubuntu@ip-172-31-38-76:~$ sudo docker pull 23012531007/house-price-app:1.0
1.0: Pulling from 23012531007/house-price-app
759a7f30c318: Pull complete
ecc510c1e359: Pull complete
0f752a1982d6: Pull complete
860d1d22e38f: Pull complete
496c1a902b3a: Pull complete
aa0e6f633e5f: Pull complete
5f5c1dcffc0a: Pull complete
a7eebd5fdbba: Pull complete
44136fa355b3: Download complete
c72fa5018c01: Download complete
Digest: sha256:a6814b0a56782396a7ca1f4d20b74167886f81acb082515ab2e84aa6b73c4b4c
Status: Downloaded newer image for 23012531007/house-price-app:1.0
docker.io/23012531007/house-price-app:1.0
```

sudo docker images

sudo docker run -d \
--name <name> \
--restart unless-stopped \
-p 5000:5000 \
<tag>:1.0

```
ubuntu@ip-172-31-38-76:~$ sudo docker images

IMAGE                                ID             DISK USAGE   CONTENT SIZE   EXTRA
23012531007/house-price-app:1.0      a6814b0a5678       727MB          165MB
ubuntu@ip-172-31-38-76:~$ sudo docker run -d --name house-price-app --restart unless-stopped -p 5000:5000 23012531007/house-price-app:1.0
baa2cd3f8429a9864299504be5f8fd56cdbd3536f9b174dcd808c3326f0f3ea7
```

sudo docker ps

```
ubuntu@ip-172-31-38-76:~$ sudo docker ps
CONTAINER ID   IMAGE                                COMMAND           CREATED         STATUS         PORTS
baa2cd3f8429   23012531007/house-price-app:1.0   "python app.py"   48 seconds ago   Up 47 seconds   0.0.0.0:5000->5000/tcp, [::]:5000->5000/tcp
```

sudo docker logs house-price-app

```
ubuntu@ip-172-31-38-76:~$ sudo docker logs house-price-app
 * Serving Flask app 'app'
 * Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on all addresses (0.0.0.0)
 * Running on http://127.0.0.1:5000
 * Running on http://172.17.0.2:5000
```

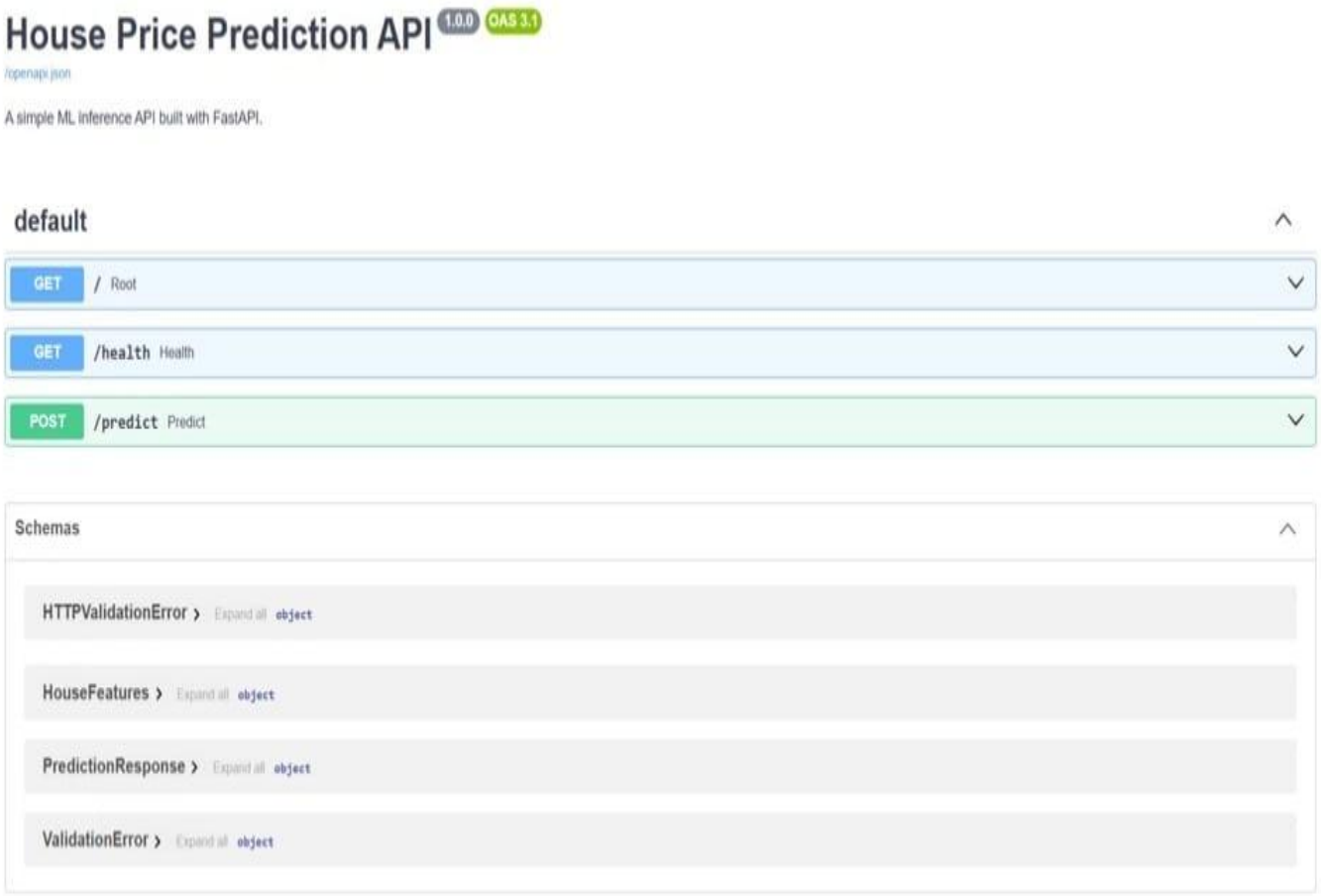
Step 8 — Verify remote access

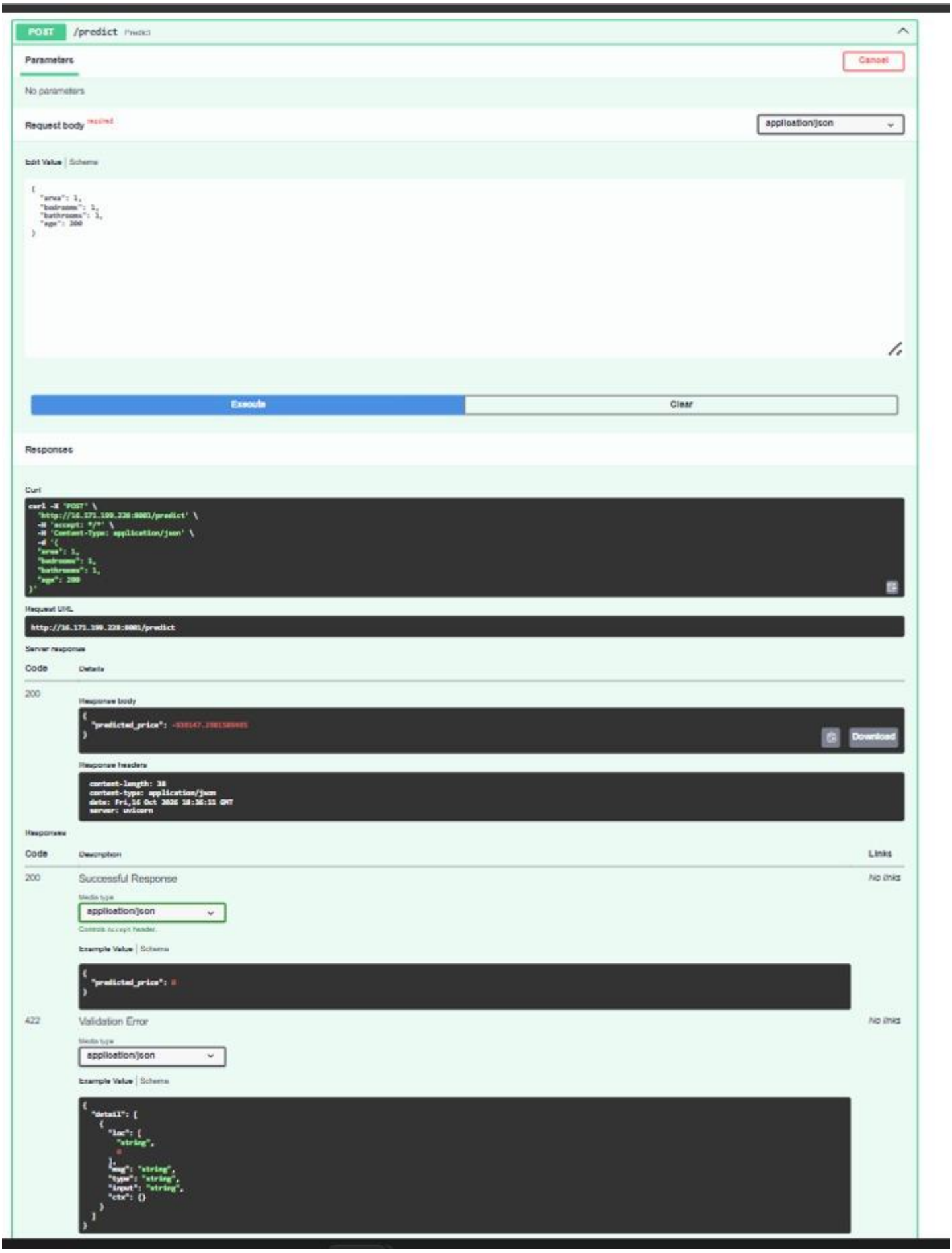
Open the following URLs in a web browser using the public IPv4 address of the AWS EC2 instance:

- **Application URL:** http://3.25.144.234:5000/
- **Health Check URL:** http://3.25.144.234:5000/health

If the application does not open, verify the EC2 instance status, security group inbound rules, and Docker container status using `sudo docker ps`. Check application logs using `sudo docker logs house-price-app` and confirm that the Flask application listens on `0.0.0.0:5000` inside the container.

The Docker container was confirmed to be running, and the application returned `HTTP 200 OK` when tested locally on the EC2 instance. External browser access still requires verification.


House Price Prediction API
1.0.0
OAS 3.1
/openapi.json
A simple ML inference API built with FastAPI.
default
GET / Root
GET /health Health
POST /predict Predict
Schemas
HTTPValidationError › Expand all object
HouseFeatures › Expand all object
PredictionResponse › Expand all object
ValidationError › Expand all object


POST /predict
Parameters
Cancel
No parameters
Request body
application/json
Edit Value | Schema
{
"area": 1,
"bedrooms": 1,
"bathrooms": 1,
"age": 200
}
Execute
Clear
Responses
Curl
curl -X 'POST' \
'http://16.171.199.220:8001/predict' \
-H 'accept: */*' \
-H 'Content-Type: application/json' \
-d '{
"area": 1,
"bedrooms": 1,
"bathrooms": 1,
"age": 200
}'
Request URL
http://16.171.199.220:8001/predict
Server response
Code
Details
200
Response body
{
"predicted_price": -338147.2981589985
}
Download
Response headers
content-length: 38
content-type: application/json
date: Fri,16 Oct 2026 18:36:11 GMT
server: uvicorn
Responses
Code
Description
Links
200
Successful Response
No links
Media type
application/json
Controls Accept header.
Example Value | Schema
{
"predicted_price": 0
}
422
Validation Error
No links
Media type
application/json
Example Value | Schema
{
"detail": [
{
"loc": [
"string",
0
],
"msg": "string",
"type": "string",
"input": "string",
"ctx": {}
}
]
}

AWS Deployment URL

Public IP Address: `3.25.144.234`

Application URL: http://3.25.144.234:5000/

Deployment Platform: Amazon Web Services (AWS) EC2

Docker Image: `23012531007/house-price-app:1.0`

Container Name: `house-price-app`

Application Port: `5000`